

▼ Import Libraries

```
# Data Handling Libraries
import pandas as pd # Used for working with datasets
import numpy as np  # Used for numerical operations and arrays.

# Data Visualization Libraries
import matplotlib.pyplot as plt # Used to create graphs and chart
import seaborn as sns # Used for attractive statistical visualiza

# Machine Learning Utilities
from sklearn.model_selection import train_test_split # Imports a

# Machine Learning Algorithms
from sklearn.linear_model import LinearRegression # Predicts v
from sklearn.ensemble import RandomForestRegressor # A powerful

# Model Evaluation Metrics
from sklearn.metrics import mean_absolute_error, r2_score # Used
```

▼ Load Dataset

```
df = pd.read_csv('train.csv') # Loads CSV file into a Pandas DataF
```

```
df.head() # Shows the first 5 rows of the dataset.
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotS
0	1	60	RL	65.0	8450	Pave	NaN	
1	2	20	RL	80.0	9600	Pave	NaN	
2	3	60	RL	68.0	11250	Pave	NaN	
3	4	70	RL	60.0	9550	Pave	NaN	
4	5	60	RL	84.0	14260	Pave	NaN	

5 rows x 81 columns

✓ Understand Dataset

```
df.shape    # Tells the size of the dataset (rows, columns)
```

```
(1460, 81)
```

```
df.columns    # See column names
```

```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea',
       'Street',
       'Alley', 'LotShape', 'LandContour', 'Utilities',
       'LotConfig',
       'LandSlope', 'Neighborhood', 'Condition1', 'Condition2',
       'BldgType',
       'HouseStyle', 'OverallQual', 'OverallCond', 'YearBuilt',
       'YearRemodAdd',
       'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd',
       'MasVnrType',
       'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation',
       'BsmtQual',
       'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1',
       'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF',
       'Heating',
       'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF',
       '2ndFlrSF',
       'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath',
       'FullBath',
       'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual',
       'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu',
       'GarageType',
       'GarageYrBlt', 'GarageFinish', 'GarageCars', 'GarageArea',
       'GarageQual',
       'GarageCond', 'PavedDrive', 'WoodDeckSF', 'OpenPorchSF',
       'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea',
       'PoolQC',
       'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold',
       'SaleType',
       'SaleCondition', 'SalePrice'],
      dtype='object')
```

✓ Select Important Features

```
# Create features
features = [
    'LotArea',
    'OverallQual',
    'OverallCond',
    'YearBuilt',
    'TotalBsmtSF',
    'GrLivArea',
    'FullBath',
    'BedroomAbvGr',
    'GarageCars'
]
```

```
# Target column
target = 'SalePrice'
```

✓ Prepare Data

```
# Selects the feature columns from the dataset.
X = df[features]
```

```
# Selects the target/output column.
y = df[target]
```

```
# Fills missing values in the dataset using the mean (average) of e
X = X.fillna(X.mean())
```

✓ Data Visualization

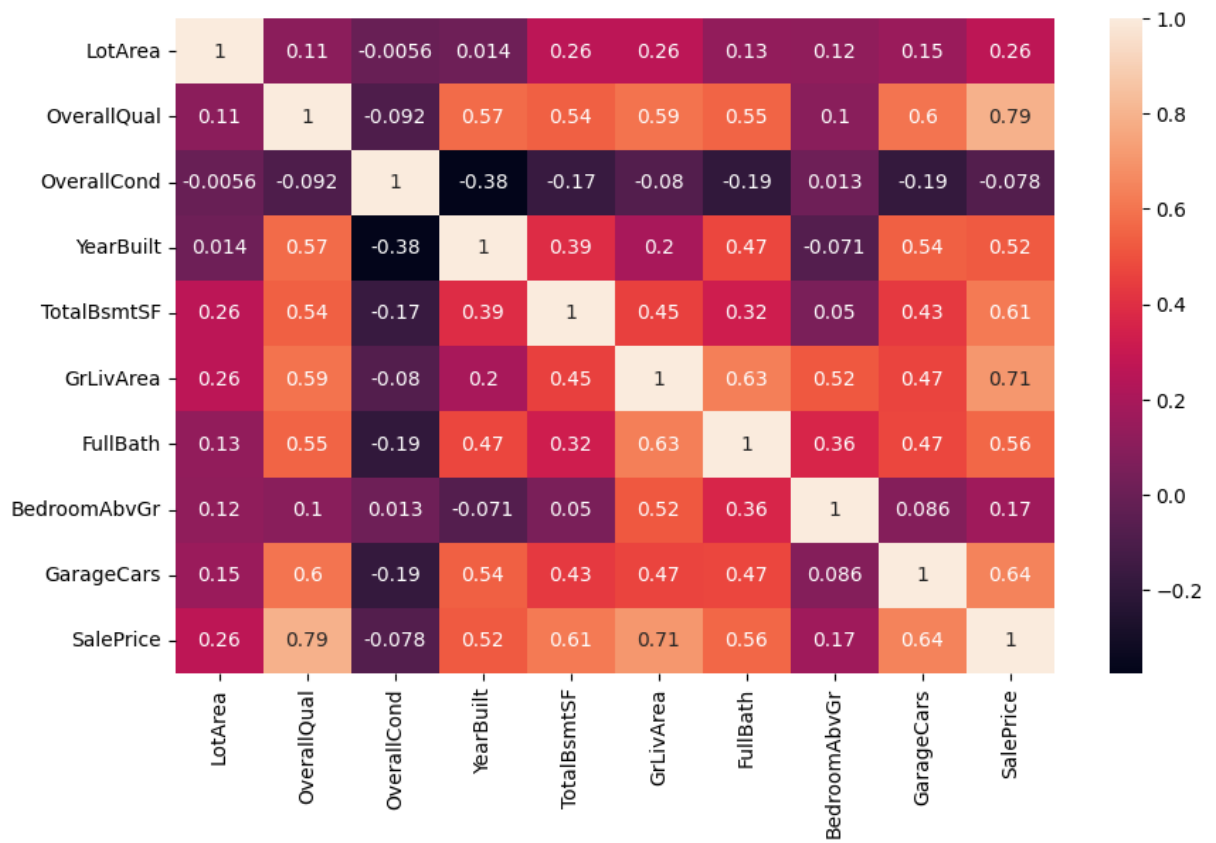
```
...
It creates a correlation heatmap to show relationships between feat
It helps you understand:
Which features strongly affect house prices
Which columns are related to each other
...

plt.figure(figsize=(10,6))

sns.heatmap(
    df[features + [target]].corr(),
    annot=True
```

)

plt.show()



Split Data

```

X_train, X_test, y_train, y_test = train_test_split(
    X, # Training feature data used to teach the model
    y, # Testing feature data used to evaluate the model
    test_size=0.2, # 20% data → testing. 80% data → training
    random_state=42 # It ensures same split every time you run the
)

```

✓ Train Linear Regression Model

''' Linear Regression predicts values using a linear relationship between:

Inputs (X)

Output (y)

Example:

Bigger house → higher price

The model tries to learn an equation like:

$$y = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n$$

Where:

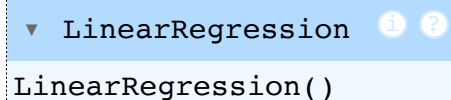
y = predicted house price

$x_1, x_2, \dots$ = features

b values = learned weights '''

```
lr_model = LinearRegression() # Creates a Linear Regression model
```

```
lr_model.fit(X_train, y_train) #Trains the model using training data
```



```
LinearRegression()
```

```
# Train Linear Regression model to predict house prices for the test set
lr_predictions = lr_model.predict(X_test)
```

✓ Evaluate Model

MAE

```
'''
MAE tells :
"On average, how far are the predictions from the real values?"
'''

# Calculate the Mean Absolute Error (MAE) of the model
mae = mean_absolute_error(y_test, lr_predictions)

print("MAE:", mae)
```

MAE: 23369.597581054506

R² Score

```
''' R2 Score tells you:
"How well does the model explain the variation in house prices?"
It measures how well predictions fit the real data.
'''

r2 = r2_score(y_test, lr_predictions)

print("R2 Score:", r2)
```

R2 Score: 0.8114463186856123

✓ Train Random Forest Model

```
'''
Random Forest is an ensemble learning algorithm.
It combines:
many Decision Trees
and averages their predictions
to improve accuracy.
'''

rf_model = RandomForestRegressor(
    n_estimators=100,    #Use 100 decision trees
    random_state=42
)

rf_model.fit(X_train, y_train)
```

▼ RandomForestRegressor ⓘ ?

RandomForestRegressor(random_state=42)

```
# Make Predictions
rf_predictions = rf_model.predict(X_test)
```

✓ Evaluate Random Forest

```
# Calculate average prediction error MAE
# Smaller MAE = better model.
rf_mae = mean_absolute_error(y_test, rf_predictions)

# Calculates how well the model explains variation in house prices
# Higher R2 = better model.
rf_r2 = r2_score(y_test, rf_predictions)

print("Random Forest MAE:", rf_mae)
print("Random Forest R2:", rf_r2)
```

```
Random Forest MAE: 17721.364103065884
Random Forest R2: 0.8990632712119048
```

“The Random Forest Regression model outperformed Linear Regression with a lower Mean Absolute Error (17,721) and a higher R² score (0.899), indicating better predictive accuracy for house price estimation.”

✓ Feature Importance

```
'''
finds and displays the importance of each feature in your Random Forest model.
Higher importance = greater influence on house price.
'''

importance = rf_model.feature_importances_ # calculates how important each feature is

# creates a table with: feature names, their importance scores
feature_importance = pd.DataFrame({
    'Feature': features,
    'Importance': importance
})

# sorts features from: highest importance to lowest importance
feature_importance.sort_values(
    by='Importance',
    ascending=False,
    inplace=True
)

print(feature_importance)
```

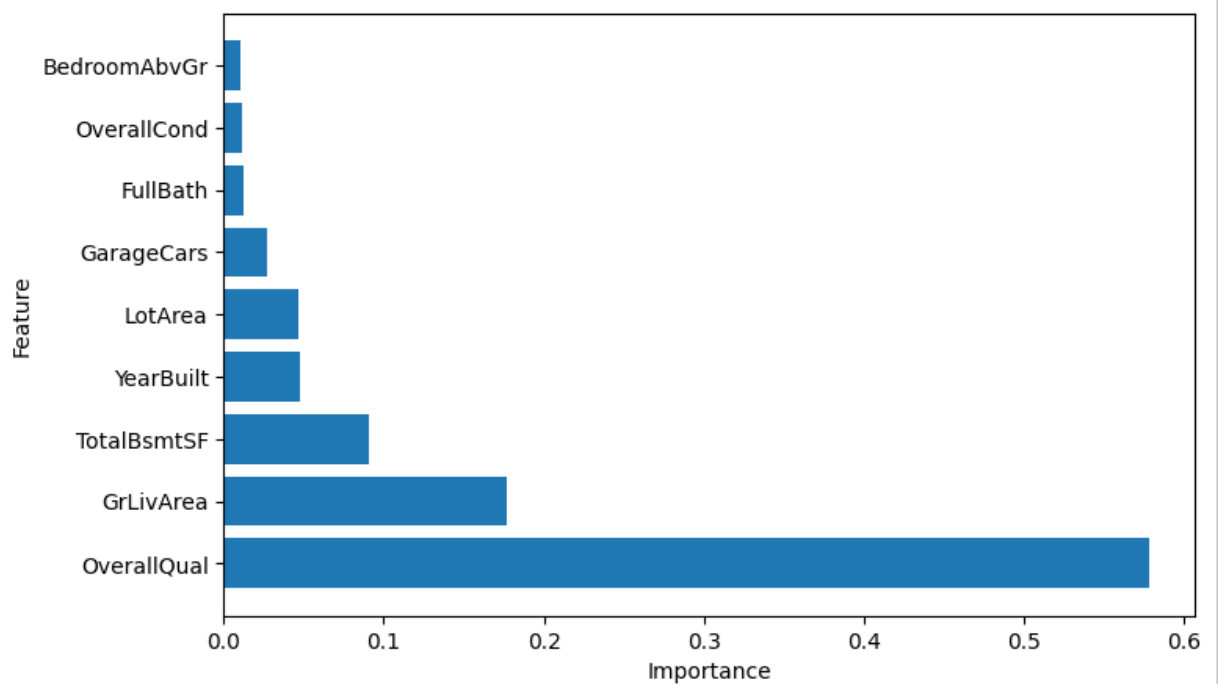
	Feature	Importance
1	OverallQual	0.578424
5	GrLivArea	0.176459
4	TotalBsmtSF	0.090486
3	YearBuilt	0.047808
0	LotArea	0.046575
8	GarageCars	0.026960
6	FullBath	0.011968
2	OverallCond	0.011203
7	BedroomAbvGr	0.010118


```
# Create Figure
plt.figure(figsize=(8,5))

# Create Horizontal Bar Chart
plt.barh(
    feature_importance['Feature'],
    feature_importance['Importance']
)

plt.xlabel("Importance")
plt.ylabel("Feature")

plt.show()
```



▼ Predict House Price

```
sample_house = pd.DataFrame([{\n    'LotArea': 8450,\n    'OverallQual': 7,\n    'OverallCond': 5,\n    'YearBuilt': 2003,\n    'TotalBsmtSF': 856,\n    'GrLivArea': 1710,\n    'FullBath': 2,\n    'BedroomAbvGr': 3,\n    'GarageCars': 2\n}])\n\nprediction = rf_model.predict(sample_house)\n\nprint("Predicted Price:", prediction[0])\n\nPredicted Price: 201701.0
```

Use More Features

✓ Categorical Encoding

Many columns in the house dataset contain text values.

Machine learning models cannot understand text directly. So we convert text into numbers.

the Kaggle dataset has many categorical columns.

- Without encoding: you use only a small subset of features, model performance stays limited.
- With encoding: model learns from much more information, accuracy improves.

```
# Find Categorical Columns
categorical_cols = df.select_dtypes(include=['object']).columns #

print(categorical_cols)
```

```
Index(['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Ut
      'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Conc
      'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1
      'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Found
      'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'Bsmt
      'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenC
      'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'G
      'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature',
      'SaleType', 'SaleCondition'],
      dtype='object')
```

```
# Apply Encoding - Use One-Hot Encoding
df_encoded = pd.get_dummies(df)
```

```
# Verify Encoding
df_encoded.head()
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt
0	1	60	65.0	8450	7	5	
1	2	20	80.0	9600	6	8	
2	3	60	68.0	11250	7	5	
3	4	70	60.0	9550	7	5	
4	5	60	84.0	14260	8	5	

5 rows x 289 columns

```
# Create x and y again Because: dataset structure changed, new enc
X = df_encoded.drop('SalePrice', axis=1)

y = df_encoded['SalePrice']
```

✓ Cross-Validation

In `train_test_split` the performance depends on one random split.

Cross-validation solves this.

It:

- splits data multiple times,
- trains/testing repeatedly,
- gives more reliable accuracy.

```
from sklearn.model_selection import cross_val_score

# Create Model
model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

# Apply Cross Validation
scores = cross_val_score(
    model,
    X,
    y,
    cv=5,    # dataset split into 5 parts, training/testing repeated
    scoring='r2'
)
```

```
print(scores)
```

```
print("Average Score:", scores.mean())
```

```
[0.87091121 0.83567295 0.87016158 0.87869736 0.80613682]
Average Score: 0.8523159859296762
```

✓ Hyperparameter Tuning

Finding the best settings automatically.

Default settings are not always best. Tuning can improve:

- accuracy,
- generalization,
- performance.

```
...
```

Grid Search automatically:

- tries many combinations of model settings
- trains models repeatedly
- finds the best-performing combination

Random Forest has parameters like:

- number of trees
- tree depth
- split conditions

Different settings give different accuracy.

Grid Search helps optimize them.

```
...
```

```
from sklearn.model_selection import GridSearchCV
```

```
# Create Parameter Grid
```

```
param_grid = {  
    'n_estimators': [100, 200], # try Random Forest with 100 trees,  
    'max_depth': [10, 20],      # Controls maximum depth of each de  
    'min_samples_split': [2, 5] # Controls maximum depth of each de  
}
```

```
# Create Model
rf = RandomForestRegressor(random_state=42)

# Apply Grid Search
grid_search = GridSearchCV( # Grid Search + Cross Validation.
    estimator=rf,
    param_grid=param_grid,
    cv=3, # Use 3-fold cross-validation.
    scoring='r2',
    n_jobs=-1 # Use all CPU cores available.
)
```

```
# Train Grid Search
grid_search.fit(X, y)

# Best Parameters
print(grid_search.best_params_)
```

```
{'max_depth': 20, 'min_samples_split': 2, 'n_estimators': 100}
```

```
# Train Final Optimized Model
best_rf = RandomForestRegressor(
    n_estimators=100,
    max_depth=20,
    min_samples_split=2,
    random_state=42
)

best_rf.fit(X_train, y_train)

best_predictions = best_rf.predict(X_test)
```

```
print("MAE:", mean_absolute_error(y_test, best_predictions))
print("R2:", r2_score(y_test, best_predictions))
```

```
MAE: 17789.916981468883
R2: 0.898076285542229
```

✓ XGBoost

A very powerful boosting algorithm. It is one of the most popular ML algorithms in: Kaggle competitions, industry projects, data science interviews.

It is Usually:

- more accurate than Random Forest,
- handles complex patterns better

```
!pip install xgboost
from xgboost import XGBRegressor
```

```
# Create Model
xgb_model = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=5
)
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/
Requirement already satisfied: numpy in /usr/local/lib/python3.12/di
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/py
Requirement already satisfied: scipy in /usr/local/lib/python3.12/di
```

```
# Train Model
xgb_model.fit(X_train, y_train)

# Predict
xgb_predictions = xgb_model.predict(X_test)

# Evaluate
print(
    r2_score(y_test, xgb_predictions)
)
```

```
0.906502902507782
```

✓ Pipeline

Earlier we manually: clean data, encode data, train model.

Pipeline automates everything.

Pipelines: reduce mistakes, make code cleaner, are industry standard.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

# Create Pipeline
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('model', RandomForestRegressor())
])

# Train Pipeline
pipeline.fit(X_train, y_train)

# Predict
predictions = pipeline.predict(X_test)
```

```
# Save Model
import joblib

joblib.dump(xgb_model, 'house_price_model.pkl') # Creates saved M
['house_price_model.pkl']
```

✓ Flask / Streamlit Deployment

Beginner project: works only inside notebook.

Deployment: turns model into real web app.


```
!pip install streamlit
```

```
Requirement already satisfied: streamlit in /usr/local/lib/python3.1
Requirement already satisfied: altair!=5.4.0,!5.4.1,<7,>=4.0 in /us
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/p
Requirement already satisfied: cachetools<8,>=5.5 in /usr/local/lib/
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/pytho
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/pyth
Requirement already satisfied: packaging>=20 in /usr/local/lib/pytho
Requirement already satisfied: pandas<4,>=1.4.0 in /usr/local/lib/py
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/p
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /usr/local/lib/
Requirement already satisfied: protobuf<8,>=3.20 in /usr/local/lib/p
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/pythor
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/p
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/pyt
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/
Requirement already satisfied: starlette>=0.40.0 in /usr/local/lib/p
Requirement already satisfied: uvicorn>=0.30.0 in /usr/local/lib/pyt
Requirement already satisfied: httptools>=0.6.3 in /usr/local/lib/py
Requirement already satisfied: anyio>=4.0.0 in /usr/local/lib/pythor
Requirement already satisfied: python-multipart>=0.0.10 in /usr/loca
Requirement already satisfied: websockets>=12.0.0 in /usr/local/lib/
Requirement already satisfied: itsdangerous>=2.1.2 in /usr/local/lib
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/c
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/pyt
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/py
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.1
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/pyt
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/pythor
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/pyth
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/loca
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/pyt
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/pyt
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/pytho
Requirement already satisfied: jsonschema-specifications>=2023.03.6
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/pyt
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12
```

```
!pip install joblib
```

```
Requirement already satisfied: joblib in /usr/local/lib/python3.12/c
```



```
%%writefile app.py

import streamlit as st
import joblib
import numpy as np

# Load trained model
model = joblib.load('house_price_model.pkl')

# Title
st.title("House Price Predictor")

# Input fields
lot_area = st.number_input("Lot Area")
overall_qual = st.number_input("Overall Quality")
overall_cond = st.number_input("Overall Condition")
year_built = st.number_input("Year Built")
total_bsmt_sf = st.number_input("Basement Area")
gr_liv_area = st.number_input("Living Area")
full_bath = st.number_input("Full Bathrooms")
bedroom = st.number_input("Bedrooms")
garage_cars = st.number_input("Garage Cars")

# Prediction button
if st.button("Predict"):

    features = np.array([
        lot_area,
        overall_qual,
        overall_cond,
        year_built,
        total_bsmt_sf,
        gr_liv_area,
        full_bath,
        bedroom,
        garage_cars
    ])

    prediction = model.predict(features)

    st.success(f"Predicted House Price: ${prediction[0]:,.2f}")
```

Overwriting app.py

```
!cat app.py
```

```
import streamlit as st
import joblib
import numpy as np

# Load trained model
model = joblib.load('house_price_model.pkl')

# Title
st.title("House Price Predictor")

# Input fields
lot_area = st.number_input("Lot Area")
overall_qual = st.number_input("Overall Quality")
overall_cond = st.number_input("Overall Condition")
year_built = st.number_input("Year Built")
total_bsmt_sf = st.number_input("Basement Area")
gr_liv_area = st.number_input("Living Area")
full_bath = st.number_input("Full Bathrooms")
bedroom = st.number_input("Bedrooms")
garage_cars = st.number_input("Garage Cars")

# Prediction button
if st.button("Predict"):

    features = np.array([
        lot_area,
        overall_qual,
        overall_cond,
        year_built,
        total_bsmt_sf,
        gr_liv_area,
        full_bath,
        bedroom,
        garage_cars
    ])

    prediction = model.predict(features)

    st.success(f"Predicted House Price: ${prediction[0]:,.2f}")
```

```

^? !!!!!
up to date, audited 23 packages in 974ms
.:
.:3 packages are looking for funding
.:  run `npm fund` for details
.:
2 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
.:

```

```
!npx localtunnel --port 8501
```

Page 21 of 21